

Fichas de Embeddings

Tokenización, Word2Vec con negative sampling, GloVe, aritmética de analogías y clasificación de AG News. 80 fichas en 8 temas.

Brandon Werner Rivera Cabrera · 23088 · Universidad del Valle de Guatemala

● Texto y tokenización

10 fichas

¿Qué es el pipeline texto → tokens → IDs → vectores?

Normalizar el texto crudo, partirlo en tokens, mapear cada token a un entero con un vocabulario, buscar su vector en una tabla de embeddings y pasar esos vectores a un modelo. Cada etapa debe ser idéntica al entrenar y al usar el modelo, o los IDs dejan de significar lo mismo.

¿Qué normalizaciones deben coincidir con GloVe para comparar de forma justa?

Minúsculas, porque GloVe 6B no distingue mayúsculas; clíticos separados como en el Penn Treebank (`do n't` , `it 's`); compuestos con guion (`well-known`); números en dígitos (`1,000` , `3.5`) y abreviaturas con punto (`mr.` , `u.s.`). Si no coinciden, palabras iguales quedan como tokens distintos y suben los fuera de vocabulario.

¿Qué son los marcadores @-@, @,@ y @.@ de WikiText?

WikiText separa con espacios los guiones y separadores dentro de palabras y números: `well @-@ known` , `1 @,@ 000` , `3 @.@ 5` . Hay que reconstruirlos como `well-known` , `1,000` y `3.5` antes de tokenizar.

¿En qué difiere un tokenizador propio por regex de NLTK Treebank?

En los casos borde: Treebank convierte las comillas en `" "` y `' '` , deja `i'd` en contracciones dobles y trata distinto las URL, los correos y números con unidades como `-5°C` . En contracciones normales, abreviaturas y guiones suelen coincidir.

¿Qué dice la ley de Zipf?

La frecuencia de una palabra es aproximadamente inversamente proporcional a su rango: $f(r) \propto 1/r^s$ con $s \approx 1$. En escala log-log se ve como una recta de pendiente cercana a -1 . Pocas palabras cubren la mayoría de los tokens y la mayoría de las palabras son raras.

● Word2Vec y SGNS

15 fichas

¿Qué predice skip-gram y qué predice CBOW?

Skip-gram predice cada palabra del contexto a partir de la central; CBOW predice la central a partir del promedio del contexto. Skip-gram genera más pares por token y representa mejor las palabras raras; CBOW es más rápido y suaviza las frecuentes.

¿Por qué word2vec no usa un softmax completo?

El softmax sobre todo el vocabulario cuesta $O(V)$ por par, con V de cientos de miles. Negative sampling cuesta $O(K)$ con K entre 5 y 20, y el softmax jerárquico $O(\log V)$.

Función objetivo de SGNS

Para un par (c, o) con K negativos n_k se maximiza $\log \sigma(u_o \cdot v_c) + \sum_k \log \sigma(-u_{n_k} \cdot v_c)$. Sube el producto con el contexto real y baja el de palabras al azar. En PyTorch se minimiza el negativo.

¿De qué distribución se muestrean los negativos?

Unigramas elevada a 0.75: $P(w) \propto \text{count}(w)^{0.75}$. El exponente aplana la distribución: las palabras raras salen más que con la unigrama pura y las muy frecuentes menos.

¿Por qué Word2Vec usa dos matrices, W y W'?

W representa la palabra cuando es centro y W' cuando es contexto o negativo. Con una sola matriz el producto de una palabra consigo misma sería alto, aunque casi nunca aparece en su propio contexto. Al final se conserva W , a veces $W + W'$.

¿Cómo se inicializan W y W' en word2vec?

W uniforme en $\pm 0.5/d$ y W' en ceros. Si las dos empezaran en cero el gradiente sería cero para siempre; con W' en cero los primeros pasos mueven sobre todo W' .

Fórmula del submuestreo

Probabilidad de conservar una aparición: $P(w) = (\sqrt{f(w)/t} + 1) \cdot t / f(w)$, con f la frecuencia relativa; es la del código C y de gensim. El paper da $P(\text{descartar}) = 1 - \sqrt{t/f}$. Con $t = 1e-4$ una palabra con el 6 % del corpus, como `the` , se descarta cerca del 96 % de las veces.

¿Por qué ayuda el submuestreo?

Las palabras muy frecuentes aparecen en casi todos los contextos y aportan poca información. Descartarlas acelera el entrenamiento, da más peso relativo a las raras y hace que la ventana alcance palabras de contenido más lejanas.

● GloVe y otros métodos

8 fichas

¿Cómo entrena GloVe?

Primero cuenta la matriz global de co-ocurrencias X con una ventana de 10 y peso 1/d por distancia. Luego ajusta $w_i \cdot w_j + b_i + b_j \approx \log X_{ij}$ por mínimos cuadrados ponderados con AdaGrad, solo sobre las celdas distintas de cero.

Función de peso de GloVe

$f(x) = (x/x_{\max})^\alpha$ si $x < x_{\max}$ y 1 en otro caso, con $x_{\max} = 100$ y $\alpha = 3/4$. Evita que los pares rarísimos, que son ruidosos, y los muy frecuentes dominen la pérdida.

¿Para qué sirve min_count ?

Descarta del vocabulario las palabras con menos de `min_count` apariciones. Tienen muy pocos contextos para aprender un buen vector, inflan la tabla y suelen ser errores o nombres raros. Subir el umbral achica el vocabulario a cambio de más tokens desconocidos.

¿Qué es un token fuera del vocabulario y cómo se maneja?

Una palabra que el modelo no vio o que quedó bajo `min_count` . Se puede descartar, mapear a `<unk>` o resolver con modelos de subpalabras como fastText o BPE, que arman un vector a partir de sus partes.

¿Para qué sirven <pad> y <unk> ?

`<pad>` rellena secuencias para que un lote tenga largo fijo y no debe aportar al resultado, por eso se usa `padding_idx` . `<unk>` agrupa las palabras fuera del vocabulario. Con `EmbeddingBag` y offsets no hace falta `<pad>` .

¿Por qué se descarta la puntuación al entrenar embeddings?

Los signos son muy frecuentes, aportan contextos de poca información y ningún benchmark de palabras los evalúa. Quitarlos elimina pares inútiles; el submuestreo los descartaría casi siempre de todas formas.

¿Por qué las ventanas no deben cruzar párrafos?

La última palabra de un párrafo o artículo y la primera del siguiente no tienen relación. word2vec y gensim tratan cada oración como unidad y solo forman pares dentro de ella.

¿Qué es la ventana dinámica?

Para cada palabra central se sortea un radio entre 1 y la ventana W . Las palabras cercanas quedan dentro más seguido, lo que pondera el contexto por distancia. El número esperado de contextos por centro es $W + 1$ en lugar de $2W$.

¿Cuántos pares genera skip-gram por época?

Con ventana fija W , unos $2W$ por token; con ventana dinámica, en promedio $W + 1$. El submuestreo elimina tokens antes de formar las ventanas, así que los pares bajan en la misma proporción que los tokens.

Efecto del tamaño de ventana

Ventanas pequeñas, como 2, capturan relaciones sintácticas y funcionales: palabras que pueden ocupar la misma posición. Ventanas grandes, como 10, capturan relaciones temáticas: palabras que aparecen en el mismo tema.

Efecto del número de negativos

Más negativos dan una señal más rica por par y ayudan en corpus pequeños, a cambio de más cómputo. Mikolov sugiere 5 a 20 en corpus pequeños y 2 a 5 en grandes. La pérdida sube con K porque suma más términos.

Efecto de la dimensión del embedding

Más dimensiones permiten codificar más relaciones, pero necesitan más datos para no quedar subentrenadas y cuestan más memoria y tiempo. Con corpus de decenas de millones de tokens, 100 a 300 suele bastar; 50 pierde capacidad.

¿Una pérdida más baja implica mejores embeddings?

No. La pérdida mide qué tan bien se separan contextos reales de negativos, no la geometría que evalúan las analogías. Cambia de escala con K , la ventana y el submuestreo, así que no se compara entre configuraciones, y dentro de una misma puede seguir bajando mientras las analogías se estancan.

¿Qué learning rate usa word2vec?

El original usa SGD con α inicial 0.025 en skip-gram, que decae linealmente hasta 0.0001 durante todo el entrenamiento. En PyTorch con mini-lotes es común usar `SparseAdam` con un lr de 0.001 a 0.01 y el mismo decaimiento.

¿Qué vectores publica GloVe?

La suma $W + \tilde{W}$ de las matrices de palabra y de contexto, que en sus experimentos daba resultados algo mejores que usar solo W .

PyTorch

10 fichas

nn.Embedding : qué hace y qué parámetros importan

Es una tabla $V \times d$ que devuelve las filas de los IDs. `num_embeddings` y `embedding_dim` definen la forma; `padding_idx` fija una fila en ceros sin gradiente; `sparse=True` genera gradientes dispersos; `max_norm` renormaliza los vectores que la superan.

¿Qué cambia con sparse=True ?

El gradiente solo contiene las filas usadas en el lote, no toda la tabla. Ahorra memoria y tiempo con vocabularios grandes. Solo lo aceptan algunos optimizadores: `SGD`, `SparseAdam` y `Adagrad`.

¿Por qué SparseAdam y no Adam con embeddings dispersos?

`Adam` no acepta gradientes dispersos. `SparseAdam` actualiza los momentos solo de las filas presentes en el lote, una actualización perezosa que es mucho más barata cuando cada lote toca una fracción mínima del vocabulario.

nn.Embedding.from_pretrained

Crea la tabla desde una matriz existente. `freeze=True`, el valor por defecto, la deja fija; `freeze=False` permite fine-tuning. También acepta `padding_idx` y `sparse`.

```
emb = nn.Embedding.from_pretrained(torch.tensor(vectores), freeze=True)
```

¿Qué ventaja tiene nn.EmbeddingBag ?

Busca y agrega los vectores de cada documento en una sola operación, sin materializar la matriz de tokens $\times d$. Es más rápido y usa menos memoria que un `Embedding` seguido de un promedio.

gensim

8 fichas

Parámetros principales de Word2Vec en gensim

`vector_size`, `window`, `min_count`, `sg` (1 skip-gram, 0 CBOW), `negative`, `sample`, `epochs`, `alpha` y `min_alpha`, `workers`. Por defecto: 100 dimensiones, ventana 5, `min_count` 5, CBOW, 5 negativos, `sample` 1e-3, 5 épocas y α 0.025.

¿Qué es KeyedVectors ?

El objeto que guarda solo las palabras y sus vectores, `model.wv`, sin el estado de entrenamiento. Tiene `most_similar`, `similarity`, `evaluate_word_analogies` y `evaluate_word_pairs`. Se guarda con `save` en `.kv` o con `save_word2vec_format` en `.txt` o `.bin`.

¿Qué hace most_similar(positive, negative) ?

Promedia los vectores unitarios de `positive` menos los de `negative`, normaliza y devuelve las palabras con mayor coseno, excluyendo las de la consulta. `most_similar(positive=["king", "woman"], negative=["man"])` es 3CosAdd.

¿Qué es most_similar_cosmul ?

La implementación de 3CosMul de Levy y Goldberg: multiplica las similitudes de los positivos, llevadas a [0, 1], y divide por las de los negativos en vez de sumarlas y restarlas.

Evaluación de embeddings

11 fichas

¿Qué es 3CosAdd?

Para «a es a b como c es a ?»: $d^* = \operatorname{argmax}_x \cos(x, b - a + c)$, con vectores normalizados y excluyendo a, b y c. Equivale a maximizar $\cos(x, b) - \cos(x, a) + \cos(x, c)$.

¿Qué es 3CosMul y por qué ayuda?

$d^* = \operatorname{argmax}_x \cos'(x, b) \cdot \cos'(x, c) / (\cos'(x, a) + \epsilon)$, con $\cos' = (\cos + 1)/2$ y $\epsilon = 0.001$. Al multiplicar, un término muy alto no compensa a otro malo, y gana menos seguido una palabra parecida solo a uno de los términos.

¿Por qué se excluyen a, b y c de la respuesta?

El vector $b - a + c$ suele quedar más cerca de b o de c que de la respuesta correcta. Sin exclusión, king - man + woman casi siempre devuelve king. La exclusión hace que la aritmética parezca más exacta de lo que es.

¿Qué contiene questions-words.txt ?

19 544 analogías en 14 categorías. 5 semánticas con 8 869 preguntas: capitales comunes y del mundo, monedas, ciudad y estado, familia. 9 sintácticas con 10 675: adjetivo a adverbio, opuestos, comparativo, superlativo, gerundio, nacionalidad, pasado, plural y plural de verbos.

¿Qué es glove-wiki-gigaword-100 ?

Vectores GloVe de 100 dimensiones entrenados sobre Wikipedia 2014 y Gigaword 5, unos 6 000 millones de tokens, con 400 000 palabras en minúsculas. `gensim` los descarga con `api.load` (128 MB).

Modos de agregación de EmbeddingBag

`sum` suma los vectores, `mean` los promedia y `max` toma el máximo por dimensión. `per_sample_weights` pondera cada token y solo funciona con `sum`.

¿Qué son los offsets de EmbeddingBag ?

Los documentos se pasan concatenados en un tensor 1D de IDs y `offsets` indica dónde empieza cada uno, así no hace falta padding. Con los documentos [1, 2, 4, 5] y [4, 3, 2, 9]: ids = [1, 2, 4, 5, 4, 3, 2, 9] y offsets = [0, 4]. Una bolsa vacía devuelve ceros.

```
bolsa = nn.EmbeddingBag(V, 100, mode="mean")
salida = bolsa(ids, offsets) # (num_docs, 100)
```

¿Cómo se congela solo la tabla de embeddings de un clasificador?

Con `modelo.emb.weight.requires_grad_(False)` y pasando al optimizador solo los parámetros con `requires_grad`. Para fine-tuning se deja en `True`, a veces con un lr menor para la tabla.

¿Cómo se escribe la pérdida de SGNS en PyTorch?

`LogSigmoid` es numéricamente estable aunque los productos sean grandes.

```
v = W(centro) # (B, d)
pos = (v * Wp(contexto)).sum(-1) # (B,)
neg = torch.bmm(Wp(negativos), v.unsqueeze(2)).squeeze(2) # (B, K)
loss = -(F.LogSigmoid(pos) + F.LogSigmoid(-neg).sum(1)).mean()
```

¿Cómo se muestrean negativos rápido en GPU?

Con la distribución acumulada de $\text{count}^{0.75}$ y `torch.searchsorted` sobre números uniformes, o con una tabla precomputada como la del código C de word2vec, de 10^8 entradas. `torch.multinomial` también sirve pero es más lento para millones de muestras.

evaluate_word_analogies y evaluate_word_pairs

El primero corre `questions-words.txt` con 3CosAdd y devuelve la accuracy por sección, usando solo las primeras `restrict_vocab` palabras, 300 000 por defecto. El segundo calcula Pearson y Spearman contra WordSim o SimLex y reporta el porcentaje de pares fuera del vocabulario.

corpus_iterable frente a corpus_file

`corpus_iterable` recibe listas de tokens, por ejemplo `LineSentence` sobre un archivo, y las reparte a los hilos desde Python. `corpus_file` lee el archivo directamente desde cada hilo y escala mejor con muchos núcleos.

¿Por qué gensim entrena rápido en CPU?

Su núcleo está en C y Cython y entrena en paralelo con hilos sin bloqueos, estilo Hogwild: cada hilo actualiza los vectores compartidos sin sincronizar y acepta colisiones ocasionales.

¿Cómo se cargan GloVe y los benchmarks con gensim?

```
import gensim.downloader as api
from gensim.test.utils import datapath
glove = api.load("glove-wiki-gigaword-100") # KeyedVectors
analogias = datapath("questions-words.txt")
```

¿Por qué evaluar todos los modelos con un vocabulario compartido?

La accuracy depende de qué preguntas se evalúan y de cuántos candidatos compiten. Con las mismas 30 000 palabras los tres modelos ven las mismas preguntas y el mismo espacio de búsqueda; con 400 000 palabras GloVe tendría más cobertura y más distractores.

WordSim-353 frente a SimLex-999

WordSim-353 mezcla similitud y relación temática: coffee y cup puntúan alto. SimLex-999 mide similitud estricta: coffee y cup puntúan bajo, igual que los antónimos. Los embeddings por ventana capturan bien la relación y peor la similitud, por eso sus correlaciones son más bajas en SimLex.

¿Por qué se usa Spearman y no Pearson?

Importa que el orden de las similitudes coincida con el orden humano, no que la relación sea lineal. Los cosenos y las puntuaciones humanas de 0 a 10 tienen escalas distintas.

¿Qué es la cobertura de un benchmark?

La fracción de preguntas o pares cuyas palabras están todas en el vocabulario. Se reporta junto con la métrica: una accuracy alta sobre pocas preguntas no es comparable con otra sobre todas.

¿Qué mide el paralelismo de los vectores diferencia?

Si una relación fuera un desplazamiento constante, man → woman, king → queen y boy → girl tendrían el mismo vector diferencia. La similitud coseno promedio entre esos vectores mide qué tan paralelos son: cerca de 1 sería aritmética exacta y cerca de 0, como pares al azar.

● Clasificación de texto

10 fichas

Baseline TF-IDF + regresión logística

Cada documento es un vector disperso con el peso tf-idf de cada palabra o bigrama, y una regresión logística multiclase separa las clases. Es rápido y muy fuerte en clasificación temática como AG News, donde unas pocas palabras clave deciden la clase.

¿Qué hace subLinear_tf ?

Reemplaza la frecuencia del término por 1 + log(tf), así una palabra repetida muchas veces no domina el vector del documento.

¿Cómo funciona el modelo EmbeddingBag + MLP?

Promedia los vectores de las palabras del documento y pasa el resultado por un MLP. Ignora el orden de las palabras igual que TF-IDF, pero trabaja en un espacio denso donde palabras parecidas tienen vectores parecidos.

¿Cuándo conviene congelar los embeddings preentrenados?

Con pocos datos: hay menos parámetros entrenables y menos sobreajuste, y las palabras que no aparecen en el entrenamiento conservan un vector útil. También cuando el dominio se parece al del preentrenamiento.

¿Cuándo conviene el fine-tuning de la tabla?

Con suficientes datos etiquetados: la tabla se adapta a la tarea, por ejemplo separando palabras que en el corpus general eran vecinas pero indican clases distintas. Con pocos datos puede sobreajustar y mover las palabras vistas lejos de las que no se vieron.

● Mi laboratorio

8 fichas

¿Por qué un subconjunto de 25 M tokens y no todo WikiText-103?

El enunciado pide al menos 20 M; 25 M deja margen y permite correr 13 configuraciones de SGNS más gensim en una sola noche. El subconjunto termina en un límite de artículo y es el mismo para SGNS y gensim.

¿Cómo se generan los pares de skip-gram en la GPU?

En cada época se sortea el submuestreo de todos los tokens y un radio por centro. Para cada distancia d de 1 a W se comparan los arreglos desplazados d posiciones y se forman los pares que están en la misma oración y dentro del radio. Luego se barajan con randperm, todo vectorizado.

¿Cómo se garantiza que gensim use el mismo corpus y vocabulario?

El corpus ya filtrado se escribe en un archivo que gensim lee con LineSentence y el mismo min_count. Como todas las palabras del archivo superan el umbral, el vocabulario queda idéntico, y el notebook lo comprueba con un assert.

¿Cómo se elige la configuración final de SGNS?

Cada iteración cambia un hiperparámetro respecto a la base. Para ventana, negativos, submuestreo y learning rate se toma el valor con mayor promedio entre la accuracy de analogías y la correlación en WordSim-353, en dimensión 100. SimLex-999 queda reservado para la evaluación final.

Cifras clave

19 544 analogías de questions-words: 8 869 semánticas y 10 675 sintácticas	0.75 exponente de la distribución de negativos	1e-3 a 1e-5 rango usual del umbral de submuestreo t	400 000 palabras de GloVe 6B, entrenado con 6 000 M tokens
353 y 999 pares de WordSim-353 y SimLex-999	0(K) frente a 0(V) costo por par de negative sampling y de softmax completo	ε = 0.001 constante de 3CosMul	W + 1 contextos esperados por centro con ventana dinámica W

Valores por defecto de gensim 4.4 y cifras de los papers de Mikolov et al. (2013), Pennington et al. (2014) y Levy y Goldberg (2014). Las preguntas sobre resultados de tus experimentos se confirman con las tablas del notebook.

¿Es king – man + woman = queen una igualdad?

No. queen suele ser la más cercana solo después de excluir las palabras de la consulta, con un coseno lejos de 1, y los vectores diferencia de una misma relación tienen cosenos bastante menores que 1. Es una regularidad aproximada.

¿Por qué los embeddings preentrenados ayudan más con pocos datos?

Traen conocimiento de millones o miles de millones de tokens: palabras que no aparecen en un entrenamiento pequeño ya tienen vectores cercanos a sus sinónimos. Con todos los datos, un modelo aleatorio o TF-IDF aprende eso directamente de las etiquetas.

¿Qué es AG News?

Noticias con título y descripción en 4 clases balanceadas: World, Sports, Business y Sci/Tech. Tiene 120 000 de entrenamiento, 30 000 por clase, y 7 600 de test, 1 900 por clase.

¿Qué limpieza adicional necesita AG News?

Trae entidades HTML sin ampersand como #39; para el apóstrofo y #36; para el dólar, además de barras invertidas. Se convierten antes de tokenizar con el mismo tokenizador del corpus de embeddings.

¿Por qué importa el porcentaje de tokens fuera del vocabulario?

Una palabra OOV no tiene vector preentrenado y, con la tabla congelada, no aporta nada. Un corpus de 25 M tokens deja fuera más palabras de noticias que GloVe con 400 000 palabras, lo que limita al modelo propio aunque sus vectores sean buenos.

¿Cuántos parámetros entrena cada variante del clasificador?

Congelado: solo el MLP, 100·256 + 256 + 256·4 + 4 = 26 884. Fine-tuning y aleatorio: el MLP más V·100 de la tabla, que con decenas de miles de palabras domina el total.

¿Por qué SGNS propio y gensim pueden diferir con los mismos hiperparámetros?

Por el optimizador: gensim usa SGD por par con hilos Hogwild y α de 0.025 a 0.0001; la implementación propia usa mini-lotes de 16 384 pares con SparseAdam. También cambian las semillas y el orden de los pares.

¿Cómo se arma el vocabulario compartido de evaluación?

Son las 30 000 palabras más frecuentes del corpus propio que también están en GloVe. El vocabulario de gensim es idéntico al propio, así que los tres modelos ven las mismas preguntas y los mismos candidatos.

¿Qué vocabulario usa el clasificador de AG News?

Las palabras con al menos 2 apariciones en el entrenamiento de AG News. Con embeddings preentrenados solo se usan las que existen en ese conjunto y las demás se descartan del documento; el modelo aleatorio las usa todas.

¿Qué compara el experimento de fracciones de datos?

El F1 de test al entrenar con 1, 5, 10, 25, 50 y 100 % de AG News para TF-IDF, embeddings aleatorios y SGNS y GloVe congelados o con fine-tuning. Las épocas crecen al reducir los datos para que todos reciban una cantidad de pasos parecida.